



Programming Fundamentals

Iteration – Quick Revision

Programming Fundamentals Team

SETU

2026-10-06





0.1 Before We Start for Loops...

0.1 Before We Start for Loops...

Last time we introduced **iteration** and worked with `while` loops.

Before moving on, let's remind ourselves of the main ideas.

The important thing is not remembering code off by heart — it is understanding **how the loop works**.



0.2 What Is Iteration?

0.2 What Is Iteration?

Iteration means repeating a set of instructions.

We often call this a **loop**.

```
number = 1
```

```
while number <= 5:  
    print(number)  
    number += 1
```



0.2 What Is Iteration?

0.2 What Is Iteration?

Iteration means repeating a set of instructions.

We often call this a **loop**.

```
number = 1
```

```
while number <= 5:  
    print(number)  
    number += 1
```

Question: Why is this better than writing five `print()` statements?



0.3 Sequence, Selection and Iteration

0.3 Sequence, Selection and Iteration

We now have three fundamental ways of controlling a program.

Sequence

Do instructions in order.

Selection

Choose what happens.

Iteration

Repeat instructions.



0.3 Sequence, Selection and Iteration

0.3 Sequence, Selection and Iteration

We now have three fundamental ways of controlling a program.

Sequence

Do instructions in order.

```
if age >= 18:  
    print("Adult")
```

Selection

Choose what happens.

Iteration

Repeat instructions.

Which one is this?



0.4 How Does a while Loop Work?

0.4 How Does a while Loop Work?

A while loop repeats **while its condition is True**.

```
count = 1
```

```
while count <= 5:  
    print(count)  
    count += 1
```

There are four important parts:

1. **Initialise** — `count = 1`
2. **Test** — `count <= 5`
4. **Repeat** — `print(count)`
3. **Update** — `count += 1`



0.5 Predict Before You Run

0.5 Predict Before You Run

What will this code display?

```
number = 2

while number <= 10:
    print(number)
    number += 2
```



0.5 Predict Before You Run



0.5 Predict Before You Run

0.5 Predict Before You Run

What will this code display?

```
number = 2
```

```
while number <= 10:  
    print(number)  
    number += 2
```

```
2  
4  
6  
8  
10
```



0.5 Predict Before You Run

Get into the habit of predicting what code will do **before** running it.



0.6 Trace the Loop

0.6 Trace the Loop

```
x = 3

while x <= 9:
    print(x)
    x += 3
```

x	x <= 9	Output
3	True	3
6	True	6
9	True	9
12	False	-

Tracing helps us see how the value changes on every iteration.



0.7 What Could Go Wrong?

0.7 What Could Go Wrong?

Look carefully at this code:

```
count = 1

while count <= 5:
    print(count)
```



0.8 What Could Go Wrong?

0.8 What Could Go Wrong?

Look carefully at this code:

```
count = 1

while count <= 5:
    print(count)
```

What is missing?



0.8 What Could Go Wrong?

0.8 What Could Go Wrong?

Look carefully at this code:

```
count = 1

while count <= 5:
    print(count)
```

What is missing?

count never changes.

The condition remains True, so the loop never ends.

This is an infinite loop.



0.9 while Is Condition-Controlled

0.9 while Is Condition-Controlled

A while loop is useful when repetition depends on a **condition**.

```
password = input("Password: ")

while password != "python":
    print("Incorrect")
    password = input("Password: ")

print("Access granted")
```

We don't necessarily know beforehand how many attempts the user will need.

Keep going **while** something is true.



0.10 Loops Can Use Selection

0.10 Loops Can Use Selection

A loop can contain an if statement.

```
number = 1
```

```
while number <= 10:  
    if number % 2 == 0:  
        print(number, "is even")  
  
    number += 1
```



0.11 Loops Can Use Selection

0.11 Loops Can Use Selection

A loop can contain an if statement.

```
number = 1

while number <= 10:
    if number % 2 == 0:
        print(number, "is even")

    number += 1
```

The loop decides **how many times** we repeat.

The if decides **what happens during each repetition**.

0.12 Validation

0.12 Validation

If we want to ensure a user enters a whole positive number

```
age_input = input("Enter your age: ")

while not age_input.isdigit():
    print("Please enter a whole number.")
    age_input = input("Enter your age: ")

age = int(age_input)

print("Your age is", age)
```



0.13 Bringing Everything Together

0.13 Bringing Everything Together

```
def count_up(limit):  
    number = 1  
  
    while number <= limit:  
        if number % 2 == 0:  
            print(number, "is even")  
        else:  
            print(number, "is odd")  
  
        number += 1  
  
count_up(5)
```

This uses:



0.13 Bringing Everything Together

- a **function**
- a **parameter**
- a **variable**
- **selection**
- **iteration**

New programming concepts don't replace the old ones — we combine them.



0.14 So Why Do We Need Another Loop?

0.14 So Why Do We Need Another Loop?

Suppose we want to do something for the numbers 1 to 5.

We can write:

```
number = 1
```

```
while number <= 5:  
    print(number)  
    number += 1
```

It works — but we had to manage number ourselves.



0.15 So Why Do We Need Another Loop?

0.15 So Why Do We Need Another Loop?

Suppose we want to do something for the numbers 1 to 5.

We can write:

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

It works — but we had to manage number ourselves.

What if Python could take care of moving through the values for us?

That's where the for loop comes in.